

Next Greater Element

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
/* Function to find the next greater
```

```
element for each element in the array */
```

```
vector<int> nextLargerElement(vector<int> arr) {
```

```
    int n = arr.size(); // size of array
```

```
    // To store the next greater elements
```

```
    vector<int> ans(n, -1);
```

```
    for(int i=0; i < n; i++) {
```

```
        // Get the current element+
```

```
        int currEle = arr[i];
```

```
        /* Nested loop to get the
```

```
        next greater element */
```

```
        for(int j=i+1; j < n; j++) {
```

```
// If the next greater element is found  
if(arr[j] > currEle) {  
  
    // Store the next greater element  
    ans[i] = arr[j];  
  
    // Break from the loop  
    break;  
}  
}  
}  
  
    // Return the answer  
    return ans;  
}  
};
```

```
int main() {  
    int n = 4;  
    vector<int> arr = {1, 3, 2, 4};  
  
    /* Creating an instance of  
    Solution class */
```

Solution sol;

**/* Function call to find the next greater element
for each element in the circular array */**

vector<int> ans = sol.nextLargerElement(arr);

cout << "The next greater elements are: ";

for(int i=0; i < n; i++) {

cout << ans[i] << " ";

}

return 0;

}

#include <bits/stdc++.h>

using namespace std;

class Solution {

public:

/* Function to find the next greater

element for each element in the array */

vector<int> nextLargerElement(vector<int> arr) {

int n = arr.size(); //size of array

// To store the next greater elements

vector<int> ans(n);

// Stack to get elements in LIFO fashion

stack<int> st;

// Start traversing from the back

for(int i=n-1; i >= 0; i--) {

// Get the current element

int currEle = arr[i];

/* Pop the elements in the stack until

the stack is not empty and the top

element is not the greater element */

while(!st.empty() && st.top() <= currEle) {

st.pop();

}

/* If the greater element is not

found, stack will be empty */

if(st.empty())

ans[i] = -1;

```
// Else store the answer  
else  
    ans[i] = st.top();  
  
/* Push the current element in the stack  
maintaining the decreasing order */  
    st.push(currEle);  
}  
  
// Return the result  
return ans;  
}  
};
```

```
int main() {  
    int n = 4;  
    vector<int> arr = {1, 3, 2, 4};  
  
/* Creating an instance of  
Solution class */  
Solution sol;  
  
/* Function call to find the next greater
```

```

        element for each element in the array */
        vector<int> ans = sol.nextLargerElement(arr);

        cout << "The next greater elements are: ";
        for(int i=0; i < n; i++) {
            cout << ans[i] << " ";
        }

        return 0;
    }
}

Next Greater Element - 2
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:

    /* Function to find the next greater element
    for each element in the circular array */
    vector<int> nextGreaterElements(vector<int> arr) {

        int n = arr.size(); // size of array

        // To store the next greater elements

```

```
vector<int> ans(n, -1);

for(int i=0; i < n; i++) {

    // Get the current element
    int currEle = arr[i];

    /* Nested loop to get the
next greater element */
    for(int j=1; j < n; j++) {

        // Getting the hypothetical index
        int ind = (j+i) % n;

        // If the next greater element is found
        if(arr[ind] > currEle) {

            // Store the next greater element
            ans[i] = arr[ind];

            // Break from the loop
            break;
        }
    }
}
```

```

    }

    // Return the answer
    return ans;
}

};

int main() {
    int n = 6;
    vector<int> arr = {5, 7, 1, 7, 6, 0};

    /* Creating an instance of
    Solution class */
    Solution sol;

    /* Function call to find the next greater element
    for each element in the circular array */
    vector<int> ans = sol.nextGreaterElements(arr);

    cout << "The next greater elements are: ";
    for(int i=0; i < n; i++) {
        cout << ans[i] << " ";
    }
}

```



```
        return 0;
    }

#include <bits/stdc++.h>

using namespace std;

class Solution {
public:

    /* Function to find the next greater
    element for each element in the array */
    vector<int> nextGreaterElements(vector<int> arr) {

        int n = arr.size(); //size of array

        // To store the next greater elements
        vector<int> ans(n);

        // Stack to get elements in LIF fashion
        stack<int> st;

        // Start traversing from the back
        for(int i = 2*n-1; i >= 0; i--) {

            // Get the actual index
```

```
int ind = i % n;

// Get the current element

int currEle = arr[ind];

/* Pop the elements in the stack until
the stack is not empty and the top
element is not the greater element */
while(!st.empty() && st.top() <= currEle) {
    st.pop();
}

// Store the answer for the second half
if(i < n) {

    /* If the greater element is not
found, stack will be empty */
if(st.empty())
    ans[i] = -1;

    // Else store the answer
else
    ans[i] = st.top();
}
```

```

        /* Push the current element in the stack
maintaining the decreasing order */
st.push(currEle);
}

// Return the result
return ans;
}
};

int main() {
int n = 6;
vector<int> arr = {5, 7, 1, 7, 6, 0};

/* Creating an instance of
Solution class */
Solution sol;

/* Function call to find the next greater
element for each element in the array */
vector<int> ans = sol.nextGreaterElements(arr);

cout << "The next greater elements are: ";

```

```
for(int i=0; i < n; i++) {  
    cout << ans[i] << " ";  
}  
  
return 0;  
}
```

Asteroid Collision

```
#include <bits/stdc++.h>  
using namespace std;
```

```
class Solution {  
public:
```

```
/* Function to determine the state of  
asteroids after all collisions */  
vector<int> asteroidCollision(vector<int> &asteroids){
```

```
// Size of the array
```

```
int n = asteroids.size();
```

```
// List implementation of stack
```

```
vector<int> st;
```

```
// Traverse all the asteroids
```

```

for(int i=0; i < n; i++) {

    /* Push the asteroid in stack if a
right moving asteroid is seen */
if(asteroids[i] > 0) {
        st.push_back(asteroids[i]);
}

    /* Else if the asteroid is moving
right, perform the collisions */
else {

        /* Until the right moving asteroids are
smaller in size, keep on destroying them */
while(!st.empty() && st.back() > 0 &&
        st.back() < abs(asteroids[i])) {

            // Destroy the asteroid
st.pop_back();
}

        /* If there is right moving asteroid
which is of same size */
if(!st.empty() &&

```

```

        st.back() == abs(asteroids[i])) {

            // Destroy both the asteroids
            st.pop_back();
        }

        /* Otherwise, if there is no left
        moving asteroid, the right moving
        asteroid will not be destroyed */
        else if(st.empty() ||
                st.back() < 0){

            // Storing the array in final state
            st.push_back(asteroids[i]);
        }
    }
}

// Return the final state of asteroids
return st;
}

};

int main() {

```

```
vector<int> arr = {10, 20, -10};
```

```
/* Creating an instance of
```

```
Solution class */
```

```
Solution sol;
```

```
/* Function call to determine the state of
```

```
asteroids after all collisions */
```

```
vector<int> ans = sol.asteroidCollision(arr);
```

```
cout << "The state of asteroids after collisions is: ";
```

```
for(int i=0; i < ans.size(); i++) {
```

```
    cout << ans[i] << " ";
```

```
}
```

```
return 0;
```

```
}
```

Sum of Subarray Minimums

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
/* Function to find the sum of the  
minimum value in each subarray */  
int sumSubarrayMins(vector<int> &arr) {  
  
    // Size of array  
    int n = arr.size();  
  
    int mod = 1e9 + 7; // Mod value  
  
    // To store the sum  
    int sum = 0;  
  
    // Traverse on the array  
    for(int i=0; i < n; i++) {  
  
        // To store the minimum of subarray  
        int mini = arr[i];  
  
        /* Nested loop to get all  
subarrays starting from index i */  
        for(int j=i; j < n; j++) {  
  
            // Update the minimum value  
            mini = min(mini, arr[j]);
```



```

        // Update the sum

        sum = (sum + mini) % mod;
    }
}

// Return the computed sum
return sum;
}
};

int main() {
    vector<int> arr = {3, 1, 2, 5};

    /* Creating an instance of
Solution class */
Solution sol;

    /* Function call to find the sum of the
minimum value in each subarray */
int ans = sol.sumSubarrayMins(arr);

    cout << "The sum of minimum value in each subarray is:
" << ans;

```

```
return 0;

}

#include <bits/stdc++.h>

using namespace std;

class Solution {

private:

    /* Function to find the indices of
next smaller elements */

vector<int> findNSE(vector<int> &arr) {

    // Size of array

int n = arr.size();

    // To store the answer

vector<int> ans(n);

    // Stack

stack<int> st;

    // Start traversing from the back

for(int i = n - 1; i >= 0; i--) {
```

```

/* Pop the elements in the stack until
the stack is not empty and the top
element is not the smaller element */
while(!st.empty() && arr[st.top()] >= arr[i]){
    st.pop();
}

// Update the answer
ans[i] = !st.empty() ? st.top() : n;

/* Push the index of current
element in the stack */
st.push(i);
}

// Return the answer
return ans;
}

/* Function to find the indices of
previous smaller or equal elements */
vector<int> findPSEE(vector<int> &arr) {

    // Size of array

```

```
int n = arr.size();

// To store the answer
vector<int> ans(n);

// Stack
stack<int> st;

// Traverse on the array
for(int i=0; i < n; i++) {

    /* Pop the elements in the stack until
the stack is not empty and the top
elements are greater than the current element */
while(!st.empty() && arr[st.top()] > arr[i]){
    st.pop();
}

// Update the answer
ans[i] = !st.empty() ? st.top() : -1;

/* Push the index of current
element in the stack */
st.push(i);
```

```
}
```

```
// Return the answer
```

```
return ans;
```

```
}
```

```
public:
```

```
/* Function to find the sum of the  
minimum value in each subarray */
```

```
int sumSubarrayMins(vector<int> &arr) {
```

```
vector<int> nse =
```

```
findNSE(arr);
```

```
vector<int> psee =
```

```
findPSEE(arr);
```

```
// Size of array
```

```
int n = arr.size();
```

```
int mod = 1e9 + 7; // Mod value
```

```
// To store the sum
```

```
int sum = 0;

// Traverse on the array
for(int i=0; i < n; i++) {

    // Count of first type of subarrays
int left = i - psee[i];

    // Count of second type of subarrays
int right = nse[i] - i;

    /* Count of subarrays where
current element is minimum */
long long freq = left*right*1LL;

    // Contribution due to current element
int val = (freq*arr[i]*1LL) % mod;

    // Updating the sum
sum = (sum + val) % mod;
}

// Return the computed sum
return sum;
```

```
    }  
};
```

```
int main() {  
    vector<int> arr = {3, 1, 2, 5};  
  
    /* Creating an instance of  
    Solution class */  
    Solution sol;  
  
    /* Function call to find the sum of the  
    minimum value in each subarray */  
    int ans = sol.sumSubarrayMins(arr);  
  
    cout << "The sum of minimum value in each subarray is:  
" << ans;  
  
    return 0;  
}
```

Sum of Subarray Ranges

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
class Solution {
```

public:

```
/* Function to find the sum of  
subarray ranges in each subarray */  
long long subArrayRanges(vector<int> &arr) {  
  
    // Size of array  
    int n = arr.size();  
  
    // To store the sum  
    long long sum = 0;  
  
    // Traverse on the array  
    for(int i=0; i < n; i++) {  
  
        // To store the smallest value of subarray  
        int smallest = arr[i];  
  
        // To store the largest value of subarray  
        int largest = arr[i];  
  
        /* Nested loop to get all  
subarrays starting from index i */  
        for(int j=i; j < n; j++) {
```



```
// Update the smallest value  
smallest = min(smallest, arr[j]);  
  
// Update the largest value  
largest = max(largest, arr[j]);  
  
// Update the sum  
sum += (largest - smallest);  
}  
}  
  
// Return the computed sum  
return sum;  
}  
};  
  
int main() {  
    vector<int> arr = {1, 2, 3};  
  
/* Creating an instance of  
   Solution class */  
   Solution sol;
```

```

    /* Function call to find the sum of
subarray ranges in each subarray */
long long ans = sol.subArrayRanges(arr);

cout << "The sum of subarray ranges is: " << ans;

return 0;
}

#include <bits/stdc++.h>
using namespace std;

class Solution {
private:

    /* Function to find the indices of
next smaller elements */
vector<int> findNSE(vector<int> &arr) {

        // Size of array
int n = arr.size();

        // To store the answer
vector<int> ans(n);

        // Stack

```

```
stack<int> st;
```

```
// Start traversing from the back
```

```
for(int i = n - 1; i >= 0; i--) {
```

```
    // Get the current element
```

```
    int currEle = arr[i];
```

```
    /* Pop the elements in the stack until
```

```
    the stack is not empty and the top
```

```
    element is not the smaller element */
```

```
    while(!st.empty() && arr[st.top()] >= currEle){
```

```
        st.pop();
```

```
    }
```

```
// Update the answer
```

```
ans[i] = !st.empty() ? st.top() : n;
```

```
/* Push the index of current
```

```
element in the stack */
```

```
st.push(i);
```

```
}
```

```
// Return the answer
```

```
    return ans;  
}
```

```
/* Function to find the indices of  
next greater elements */
```

```
vector<int> findNGE(vector<int> &arr) {
```

```
    // Size of array
```

```
    int n = arr.size();
```

```
    // To store the answer
```

```
    vector<int> ans(n);
```

```
    // Stack
```

```
    stack<int> st;
```

```
    // Start traversing from the back
```

```
    for(int i = n - 1; i >= 0; i--) {
```

```
        // Get the current element
```

```
        int currEle = arr[i];
```

```
        /* Pop the elements in the stack until  
        the stack is not empty and the top
```

```

        element is not the greater element */
        while(!st.empty() && arr[st.top()] <= currEle){
            st.pop();
        }

        // Update the answer
        ans[i] = !st.empty() ? st.top() : n;

        /* Push the index of current
        element in the stack */
        st.push(i);
    }

    // Return the answer
    return ans;
}

/* Function to find the indices of
previous smaller or equal elements */
vector<int> findPSEE(vector<int> &arr) {

    // Size of array
    int n = arr.size();

```

// To store the answer

vector<int> ans(n);

// Stack

stack<int> st;

// Traverse on the array

for(int i=0; i < n; i++) {

// Get the current element

int currEle = arr[i];

/* Pop the elements in the stack until

the stack is not empty and the top

elements are greater than the current element */

while(!st.empty() && arr[st.top()] > currEle){

st.pop();

}

// Update the answer

ans[i] = !st.empty() ? st.top() : -1;

/* Push the index of current

element in the stack */

```
        st.push(i);
    }

    // Return the answer
    return ans;
}

/* Function to find the indices of
previous greater or equal elements */
vector<int> findPGEE(vector<int> &arr) {

    // Size of array
    int n = arr.size();

    // To store the answer
    vector<int> ans(n);

    // Stack
    stack<int> st;

    // Traverse on the array
    for(int i=0; i < n; i++) {

        // Get the current element
```

```
int currEle = arr[i];
```

```
/* Pop the elements in the stack until  
the stack is not empty and the top  
elements are smaller than the current element */
```

```
while(!st.empty() && arr[st.top()] < currEle){  
    st.pop();  
}
```

```
// Update the answer
```

```
ans[i] = !st.empty() ? st.top() : -1;
```

```
/* Push the index of current  
element in the stack */
```

```
st.push(i);
```

```
}
```

```
// Return the answer
```

```
return ans;
```

```
}
```

```
/* Function to find the sum of the  
minimum value in each subarray */
```

```
long long sumSubarrayMins(vector<int> &arr) {
```



```
vector<int> nse = findNSE(arr);
```

```
vector<int> psee = findPSEE(arr);
```

```
// Size of array
```

```
int n = arr.size();
```

```
// To store the sum
```

```
long long sum = 0;
```

```
// Traverse on the array
```

```
for(int i=0; i < n; i++) {
```

```
    // Count of first type of subarrays
```

```
    int left = i - psee[i];
```

```
    // Count of second type of subarrays
```

```
    int right = nse[i] - i;
```

```
    /* Count of subarrays where
```

```
    current element is minimum */
```

```
    long long freq = left*right*1LL;
```

```

// Contribution due to current element

long long val = (freq*arr[i]*1LL);


// Updating the sum

sum += val;
}


// Return the computed sum

return sum;
}


/* Function to find the sum of the
maximum value in each subarray */
long long sumSubarrayMaxs(vector<int> &arr) {


vector<int> nge = findNGE(arr);


vector<int> pgee = findPGEE(arr);


// Size of array

int n = arr.size();


// To store the sum

long long sum = 0;

```

```
// Traverse on the array
for(int i=0; i < n; i++) {

    // Count of first type of subarrays
    int left = i - pgee[i];

    // Count of second type of subarrays
    int right = nge[i] - i;

    /* Count of subarrays where
    current element is minimum */
    long long freq = left*right*1LL;

    // Contribution due to current element
    long long val = (freq*arr[i]*1LL);

    // Updating the sum
    sum += val;
}

// Return the computed sum
return sum;
}
```

public:

```
/* Function to find the sum of  
subarray ranges in each subarray */  
long long subArrayRanges(vector<int> &arr) {  
  
    // Return the result  
    return ( sumSubarrayMaxs(arr) -  
            sumSubarrayMins(arr) );  
}  
};
```

```
int main() {  
    vector<int> arr = {1, 2, 3};  
  
    /* Creating an instance of  
    Solution class */  
    Solution sol;  
  
    /* Function call to find the sum of  
    subarray ranges in each subarray */  
    long long ans = sol.subArrayRanges(arr);  
  
    cout << "The sum of subarray ranges is: " << ans;
```

```
    return 0;
}
```

Remove K Digits

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    /* Function to find the smallest possible  
    integer after removing k digits */
```

```
    string removeKdigits(string nums, int k) {
```

```
        stack <char> st; // Stack
```

```
        // Traverse on the given string
```

```
        for(int i=0; i < nums.size(); i++) {
```

```
            // Current digit
```

```
            char digit = nums[i];
```

```
            /* Pop last digits (when possible)
```

```
            if a smaller digit is found*/
```

```
while(!st.empty() && k > 0  
    && st.top() > digit) {  
  
    st.pop(); // Pop the last digit  
    k--; // Decrement K by 1  
}  
  
// Push the current digit  
st.push(digit);  
}  
  
// If more digits can be removed  
while(!st.empty() && k > 0) {  
  
    st.pop(); // Pop the last added digits  
    k--; // Decrement K by 1  
}  
  
// Handling edge case  
if(st.empty()) return "0";  
  
// To store the result  
string res = "";
```

```
// Adding digits in stack to result  
while(!st.empty()) {  
    res.push_back(st.top());  
    st.pop();  
}  
  
// Trimming the zeroes at the back  
while(res.size() > 0 &&  
    res.back() == '0') {  
  
    res.pop_back();  
}  
  
// Reverse to get the actual number  
reverse(res.begin(), res.end());  
  
// Edge case  
if(res.empty()) return "0";  
  
// Return the stored result  
return res;  
}  
};
```

```

int main() {
    string nums = "541892";
    int k = 2;

    /* Creating an instance of
Solution class */
    Solution sol;

    /* Function call to find the smallest
possible integer after removing k digits */
    string ans = sol.removeKdigits(nums, k);

    cout << "The smallest possible integer after removing k
digits is: " << ans;

    return 0;
}

```

Implement Min Stack

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Class to implement Minimum Stack
```

```
class MinStack {
```

```
private:
```


// Initialize a stack

stack <pair<int,int>> st;

public:

// Empty Constructor

MinStack() {

}

// Method to push a value in stack

void push(int value) {

// If stack is empty

if(st.empty()) {

// Push current value as minimum

st.push({value, value});

return;

}

// Update the current minimum

int mini = min(getMin(), value);

// Add the pair to the stack

```
    st.push({value, mini});  
}
```

```
// Method to pop a value from stack
```

```
void pop() {  
    // Using in-built pop method  
    st.pop();  
}
```

```
// Method to get the top of stack
```

```
int top() {  
    // Return the top value  
    return st.top().first;  
}
```

```
// Method to get the minimum in stack
```

```
int getMin() {  
    // Return the minimum  
    return st.top().second;  
}
```

```
};
```

```
int main() {
```

```
    MinStack s;
```

```

// Function calls

s.push(-2);

s.push(0);

s.push(-3);

cout << s.getMin() << " ";

s.pop();

cout << s.top() << " ";

s.pop();

cout << s.getMin();


return 0;
}

#include <bits/stdc++.h>

using namespace std;


// Class to implement Minimum Stack

class MinStack {

private:

    // Initialize a stack

    stack <int> st;

    // To store the minimum value

    int mini;

```

public:

// Empty Constructor

```
MinStack() {  
}
```

// Method to push a value in stack

```
void push(int value) {
```

```
    // If stack is empty
```

```
    if(st.empty()) {
```

```
        //Update the minimum value
```

```
        mini = value;
```

```
        // Push current value as minimum
```

```
        st.push( value );
```

```
        return;
```

```
    }
```

```
    // If the value is greater than the minimum
```

```
    if(value > mini) {
```

```
        st.push(value);
```

```
    }
```

```
    else {
```

```

    // Add the modified value to stack
    st.push(2 * value - mini);

    // Update the minimum
    mini = value;
}
}

// Method to pop a value from stack
void pop() {
    // Base case
    if(st.empty()) return;

    // Get the top
    int x = st.top();
    st.pop(); // Pop operation

    // If the modified value was added to stack
    if(x < mini) {
        // Update the minimum
        mini = 2 * mini - x;
    }
}

// Method to get the top of stack

```

```

int top() {
    // Base case
    if(st.empty()) return -1;

    // Get the top
    int x = st.top();

    // Returnn top if minimum is less than the top
    if(mini < x) return x;

    //Otherwise return mini
    return mini;
}

// Method to get the minimum in stack
int getMin() {
    // Return the minimum
    return mini;
}
};

int main() {
    MinStack s;

```

```
// Function calls  
s.push(-2);  
s.push(0);  
s.push(-3);  
cout << s.getMin() << " ";  
s.pop();  
cout << s.top() << " ";  
s.pop();  
cout << s.getMin();  
  
return 0;  
}
```

Sliding Window Maximum

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
// Function to get the maximum sliding window
```

```
vector<int> maxSlidingWindow(vector<int> &arr, int k) {
```

```
    int n = arr.size(); // Size of array
```

```
    // To store the answer
```

```
vector<int> ans;

/* Traverse on the array
for valid window */
for(int i=0; i <= n-k; i++) {

    // To store the maximum of the window
int maxi = arr[i];

    // Traverse the window
for(int j=i; j < i+k; j++) {
        // Update the maximum
maxi = max(maxi, arr[j]);
}

    // Add the maximum to the result
ans.push_back(maxi);
}

// Return the stored result
return ans;
}
};
```



```
int main() {  
  
    vector<int> arr = {4, 0, -1, 3, 5, 3, 6, 8};  
  
    int k = 3;  
  
    /* Creating an instance of  
  
Solution class */  
  
    Solution sol;  
  
    /* Function call to get the  
  
maximum sliding window */  
  
    vector<int> ans = sol.maxSlidingWindow(arr, k);  
  
    cout << "The maximum elements in the sliding window  
are: ";  
  
    for(int i=0; i < ans.size(); i++) {  
        cout << ans[i] << " ";  
    }  
  
    return 0;  
  
}  
  
#include <bits/stdc++.h>  
  
using namespace std;  
  
class Solution {
```

public:

// Function to get the maximum sliding window

vector<int> maxSlidingWindow(vector<int> &arr, int k) {

int n = arr.size(); // Size of array

// To store the answer

vector<int> ans;

// Deque data structure

deque <int> dq;

// Traverse the

for(int i=0; i < n; i++) {

// Update deque to maintain current window

if (!dq.empty() && dq.front() <= (i-k)) {

dq.pop_front();

}

/* Maintain the monotonic (decreasing)

order of elements in deque */

while (!dq.empty() && arr[dq.back()] <= arr[i]) {

dq.pop_back();

```

    }

    // Add current elements index to the deque
    dq.push_back(i);

    /* Store the maximum element from
    the first window possible */
    if (i >= (k-1)) {
        ans.push_back(arr[dq.front()]);
    }
}

// Return the stored result
return ans;
}
};

```

```

int main() {
    vector<int> arr = {4, 0, -1, 3, 5, 3, 6, 8};
    int k = 3;

    /* Creating an instance of
    Solution class */
    Solution sol;

```

```

/* Function call to get the
maximum sliding window */
vector<int> ans = sol.maxSlidingWindow(arr, k);

cout << "The maximum elements in the sliding window
are: ";

for(int i=0; i < ans.size(); i++) {
    cout << ans[i] << " ";
}

return 0;
}

```

Trapping Rainwater

```

#include <bits/stdc++.h>
using namespace std;

class Solution {
private:

    // Function to find the prefix maximum array
    vector<int> findPrefixMax(vector<int> &arr, int &n) {
        // To store the prefix maximum
        vector<int> prefixMax(n);
    }
}

```

```
// Initial configuration  
prefixMax[0] = arr[0];  
  
// Traverse the array  
for(int i=1; i < n; i++) {  
  
    // Store the maximum till ith index  
    prefixMax[i] =  
        max(prefixMax[i-1], arr[i]);  
}  
  
// Return the prefix maximum array  
return prefixMax;  
}  
  
// Function to find the suffix maximum array  
vector<int> findSuffixMax(vector<int> &arr, int &n) {  
  
    // To store the suffix maximum  
    vector<int> suffixMax(n);  
  
    // Initial configuration  
    suffixMax[n-1] = arr[n-1];
```

```

// Traverse the array
for(int i=n-2; i >= 0; i--) {

    // Store the maximum till ith index
    suffixMax[i] =
        max(suffixMax[i+1], arr[i]);
}

// Return the suffix maximum array
return suffixMax;
}

public:

// Function to get the trapped water
int trap(vector<int> &height){

    int n = height.size(); // Size of array

    // To store the total trapped rainwater
    int total = 0;

    // Calculate prefix maximum array
    vector<int> leftMax =
        findPrefixMax(height, n);

```

```

// Calculate suffix maximum array

vector<int> rightMax =
    findSuffixMax(height, n);


// Traverse the array
for(int i=0; i < n; i++) {


    /* If there are higher grounds
on both side to hold water */
if(height[i] < leftMax[i] &&
    height[i] < rightMax[i]) {


        // Add up the water on top of current height
total += ( min(leftMax[i], rightMax[i]) -
            height[i] );
}
}


// Return the result
return total;
}
};

```

```
int main() {  
    vector<int> heights = {4, 2, 0, 3, 2, 5};  
  
    /* Creating an instance of  
    Solution class */  
    Solution sol;  
  
    // Function call to get the trapped water  
    int ans = sol.trap(heights);  
  
    cout << "The trapped rainwater is: " << ans;  
    return 0;  
}  
  
#include <bits/stdc++.h>  
using namespace std;  
  
class Solution {  
public:  
  
    // Function to get the trapped water  
    int trap(vector<int> &height){  
  
        int n = height.size(); // Size of array
```



```
// To store the total trapped rainwater  
int total = 0;  
  
// To store the maximums on both sides  
int leftMax = 0, rightMax = 0;  
  
// Left and Right pointers  
int left = 0, right = n-1;  
  
// Traverse from both ends  
while(left < right) {  
  
    // If left height is smaller or equal  
    if(height[left] <= height[right]) {  
  
        // If water can be stored  
        if(leftMax > height[left]) {  
  
            // Update total water  
            total += leftMax - height[left];  
        }  
  
        // Else update maximum height on left  
        else leftMax = height[left];
```

```
// Shift left by 1  
left = left + 1;  
}  
  
// Else if right height is smaller  
else {  
  
// If water can be stored  
if(rightMax > height[right]) {  
  
// Update total water  
total += rightMax - height[right];  
}  
  
// Else update maximum height on right  
else rightMax = height[right];  
  
// Shift right by 1  
right = right - 1;  
}  
}  
  
// Return the result
```

```
        return total;
    }
};
```

```
int main() {
    vector<int> heights = {4, 2, 0, 3, 2, 5};

    /* Creating an instance of
    Solution class */
    Solution sol;

    // Function call to get the trapped water
    int ans = sol.trap(heights);

    cout << "The trapped rainwater is: " << ans;
    return 0;
}
```

Largest rectangle in a histogram

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    /* Function to find the indices of
```

```
next smaller elements */  
vector<int> findNSE(vector<int> &arr) {  
  
    // Size of array  
    int n = arr.size();  
  
    // To store the answer  
    vector<int> ans(n);  
  
    // Stack  
    stack<int> st;  
  
    // Start traversing from the back  
    for(int i = n - 1; i >= 0; i--) {  
  
        // Get the current element  
        int currEle = arr[i];  
  
        /* Pop the elements in the stack until  
        the stack is not empty and the top  
        element is not the smaller element */  
        while(!st.empty() && arr[st.top()] >= arr[i]){  
            st.pop();  
        }
```

```

// Update the answer

ans[i] = !st.empty() ? st.top() : n;


/* Push the index of current
element in the stack */

st.push(i);
}


// Return the answer

return ans;
}


/* Function to find the indices of
previous smaller elements */
vector<int> findPSE(vector<int> &arr) {


// Size of array

int n = arr.size();


// To store the answer

vector<int> ans(n);


// Stack

```

```
stack<int> st;
```

```
// Traverse on the array
```

```
for(int i=0; i < n; i++) {
```

```
    // Get the current element
```

```
    int currEle = arr[i];
```

```
    /* Pop the elements in the stack until  
    the stack is not empty and the top  
    elements is not the smaller element */
```

```
    while(!st.empty() && arr[st.top()] >= arr[i]){  
        st.pop();  
    }
```

```
// Update the answer
```

```
ans[i] = !st.empty() ? st.top() : -1;
```

```
/* Push the index of current  
element in the stack */
```

```
    st.push(i);  
}
```

```
// Return the answer
```

```
    return ans;  
}
```

public:

```
// Function to find the largest rectangle area  
int largestRectangleArea(vector<int> &heights) {  
  
    /* Determine the next and  
    previous smaller elements */  
    vector<int> nse = findNSE(heights);  
    vector<int> pse = findPSE(heights);  
  
    // To store largest area  
    int largestArea = 0;  
  
    // To store current area  
    int area;  
  
    // Traverse on the array  
    for(int i=0; i < heights.size(); i++) {  
  
        // Calculate current area  
        area = heights[i] * (nse[i] - pse[i] - 1);
```

```

        // Update largest area
        largestArea = max(largestArea, area);
    }

    // Return largest area found
    return largestArea;
}

};

int main() {
    vector<int> heights = {2, 1, 5, 6, 2, 3};

    /* Creating an instance of
    Solution class */
    Solution sol;

    // Function call to find the largest rectangle area
    int ans = sol.largestRectangleArea(heights);

    cout << "The largest rectangle area is: " << ans;
    return 0;
}

#include <bits/stdc++.h>

```



```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
// Function to find the largest rectangle area
```

```
int largestRectangleArea(vector<int> &heights) {
```

```
    int n = heights.size(); // Size of array
```

```
    // Stack
```

```
    stack<int> st;
```

```
    // To store largest area
```

```
    int largestArea = 0;
```

```
    // To store current area
```

```
    int area;
```

```
    /* To store the indices of next
```

```
    and previous smaller elements */
```

```
    int nse, pse;
```

```
    // Traverse on the array
```

```
for(int i=0; i < n; i++) {  
  
    /* Pop the elements in the stack until  
    the stack is not empty and the top  
    elements is not the smaller element */  
    while(!st.empty() &&  
        heights[st.top()] >= heights[i]){  
  
        // Get the index of top of stack  
        int ind = st.top();  
        st.pop();  
  
        /* Update the index of  
        previous smaller element */  
        pse = st.empty() ? -1 : st.top();  
  
        /* Next smaller element index for  
        the popped element is current index */  
        nse = i;  
  
        // Calculate the area of the popped element  
        area = heights[ind] * (nse-pse-1);  
  
        // Update the maximum area
```

```

        largestArea = max(largestArea, area);
    }

    // Push the current index in stack
    st.push(i);
}

// For elements that are not popped from stack
while(!st.empty()) {

    // NSE for such elements is size of array
    nse = n;

    // Get the index of top of stack
    int ind = st.top();
    st.pop();

    // Update the previous smaller element
    pse = st.empty() ? -1 : st.top();

    // Calculate the area of the popped element
    area = heights[ind] * (nse-pse-1);

    // Update the maximum area

```

```

        largestArea = max(largestArea, area);
    }

    // Return largest area found
    return largestArea;
}

};

int main() {
    vector<int> heights = {2, 1, 5, 6, 2, 3};

    /* Creating an instance of
    Solution class */
    Solution sol;

    // Function call to find the largest rectangle area
    int ans = sol.largestRectangleArea(heights);

    cout << "The largest rectangle area is: " << ans;
    return 0;
}

```

Maximum Rectangles

```

#include <bits/stdc++.h>

using namespace std;

```

```
class Solution {  
private:  
  
    int largestRectangleArea(vector<int> &heights) {  
        int n = heights.size();  
        stack<int> st;  
        int maxArea = 0;  
  
        for (int i = 0; i < n; i++) {  
            while (!st.empty() && heights[st.top()] >=  
heights[i]) {  
                int h = heights[st.top()];  
                st.pop();  
                int left = st.empty() ? -1 : st.top();  
                int width = i - left - 1;  
                maxArea = max(maxArea, h * width);  
            }  
            st.push(i);  
        }  
  
        while (!st.empty()) {  
            int h = heights[st.top()];  
            st.pop();  
            int left = st.empty() ? -1 : st.top();
```

```

        int width = n - left - 1;
        maxArea = max(maxArea, h * width);
    }

    return maxArea;
}

```

public:

```

    int maximalAreaOfSubMatrixOfAll1(vector<vector<int>>
    &matrix) {
        int n = matrix.size();
        int m = matrix[0].size();

        vector<int> heights(m, 0);
        int maxArea = 0;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (matrix[i][j] == 0)
                    heights[j] = 0;
                else
                    heights[j]++;
            }
        }
    }

```

```
        maxArea = max(maxArea,  
largestRectangleArea(heights));  
    }
```

```
    return maxArea;  
}
```

```
};
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    // Function to compute the maximal area of a submatrix  
    consisting of all 1s
```

```
    int maximalAreaOfSubMatrixOfAll1(vector<vector<int>>  
&matrix) {
```

```
        // If the matrix is empty, return 0 as no area can be  
        formed
```

```
        if (matrix.empty()) return 0;
```

```
        // Store the number of rows in the matrix
```

```
        int n = matrix.size();
```

```
        // Store the number of columns in the matrix
```

```
int m = matrix[0].size();
```

```
// DP table to store consecutive 1s ending at each cell  
in the row
```

```
vector<vector<int>> dp(n, vector<int>(m, 0));
```

```
// Variable to store the maximum area found
```

```
int maxArea = 0;
```

```
// Iterate over each row
```

```
for (int i = 0; i < n; i++) {
```

```
    // Iterate over each column
```

```
    for (int j = 0; j < m; j++) {
```

```
        // Check if the current cell contains 1
```

```
        if (matrix[i][j] == 1) {
```

```
            // Compute consecutive 1s ending at this cell  
in the current row
```

```
            dp[i][j] = (j == 0) ? 1 : dp[i][j - 1] + 1;
```

```
            // Initialize width as the current consecutive  
1s
```



```

        int width = dp[i][j];

        // Traverse upwards to form rectangles with
current cell as bottom-right corner
        for (int k = i; k >= 0; k--) {

            // If no 1s at this column in the upper row,
break the loop
            if (dp[k][j] == 0) break;

            // Update width as the minimum width seen
so far
            width = min(width, dp[k][j]);

            // Update maximum area using current
width and height
            maxArea = max(maxArea, width * (i - k +
1));
        }
    }
}

// Return the maximum area of submatrix of all 1s
found

```

```
        return maxArea;
    }
};
```

Stock span problem

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    // Function to find the span of stock prices for each day
```

```
    vector<int> stockSpan(vector<int> arr, int n) {
```

```
        // To store the answer
```

```
        vector<int> ans(n);
```

```
        // Traverse on the array
```

```
        for(int i=0; i < n; i++) {
```

```
            // To store the current span of stocks
```

```
            int currSpan = 0;
```

```
            // Traversing backwards to find stock span
```

```
            for(int j=i; j >= 0; j--) {
```

```

        // Update stock span
        if(arr[j] <= arr[i]) {
            currSpan++;
        }

        // Else break out from loop
        else break;
    }

    // Store the current span
    ans[i] = currSpan;
}

// Return the result
return ans;
}

};

int main() {
    int n = 7;
    vector<int> arr = {120, 100, 60, 80, 90, 110, 115};

    /* Creating an instance of
    Solution class */

```

Solution sol;

/* Function call to find the span

of stock prices for each day */

vector<int> ans = sol.stockSpan(arr, n);

cout << "The span of stock prices is: ";

for(int i=0; i < n; i++) {

cout << ans[i] << " ";

}

return 0;

}

#include <bits/stdc++.h>

using namespace std;

class Solution {

private:

/* Function to find the indices of previous

greater element for each element in the array */

vector<int> findPGE(vector<int> arr) {

int n = arr.size(); //size of array

// To store the previous greater elements

vector<int> ans(n);

// Stack to get elements in LIFO fashion

stack<int> st;

// Start traversing from the front

for(int i=0; i < n; i++) {

// Get the current element

int currEle = arr[i];

/* Pop the elements in the stack until

the stack is not empty and the top

element is not the greater element */

while(!st.empty() && arr[st.top()] <= currEle) {

st.pop();

}

/* If the greater element is not

found, stack will be empty */

if(st.empty())

ans[i] = -1;

```
// Else store the answer  
else  
    ans[i] = st.top();  
  
    // Push the current index in the stack  
    st.push(i);  
}  
  
// Return the result  
return ans;  
}
```

public:

```
// Function to find the span of stock prices for each day  
vector <int> stockSpan(vector<int> arr, int n) {
```

```
    // Get the indices of previous greater elements  
    vector<int> PGE = findPGE(arr);
```

```
    // To store the answer  
    vector<int> ans(n);
```

```
    // Compute the result
```

```

        for(int i=0; i < n; i++) {
            ans[i] = i - PGE[i];
        }

        // Return the result
        return ans;
    }
};

int main() {
    int n = 7;
    vector<int> arr = {120, 100, 60, 80, 90, 110, 115};

    /* Creating an instance of
    Solution class */
    Solution sol;

    /* Function call to find the span
    of stock prices for each day */
    vector<int> ans = sol.stockSpan(arr, n);

    cout << "The span of stock prices is: ";
    for(int i=0; i < n; i++) {
        cout << ans[i] << " ";
    }
}

```

```
}
```

```
return 0;
```

```
}
```

Celebrity Problem

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
// Function to find the index of celebrity
```

```
int celebrity(vector<vector<int>> &M){
```

```
// Size of given matrix
```

```
int n = M.size();
```

```
/* To store count of people who
```

```
know person of index i */
```

```
vector<int> knowMe(n, 0);
```

```
/* To store count of people who
```

```
the person of index i knows */
```

```
vector<int> lknow(n, 0);
```


// Traverse on given matrix

for(int i=0; i < n; i++) {

for(int j=0; j < n; j++) {

// If person i knows person j

if(M[i][j] == 1) {

knowMe[j]++;

Iknow[i]++;

}

}

}

// Traverse for all persons to find the celebrity

for(int i=0; i < n; i++) {

// Return the index of celebrity

if(knowMe[i] == n-1 && Iknow[i] == 0) {

return i;

}

}

// Return -1 if no celebrity is found

return -1;

}

};

int main() {

vector<vector<int>> M = {

{0, 1, 1, 0},

{0, 0, 0, 0},

{1, 1, 0, 0},

{0, 1, 1, 0}

};

/* Creating an instance of

Solution class */

Solution sol;

// Function call to find the index of celebrity

int ans = sol.celebrity(M);

cout << "The index of celebrity is: " << ans;

return 0;

}

#include <bits/stdc++.h>

using namespace std;

```
class Solution {  
public:  
  
    // Function to find the index of celebrity  
int celebrity(vector<vector<int>> &M){  
  
    // Size of given matrix  
int n = M.size();  
  
    // Top and Down pointers  
int top = 0, down = n-1;  
  
    // Traverse for all the people  
while(top < down) {  
  
        /* If top knows down,  
        it can not be a celebrity */  
        if(M[top][down] == 1) {  
            top = top + 1;  
        }  
  
        /* If down knows top,  
        it can not be a celebrity */  
        else if(M[down][top] == 1) {  
            down = down - 1;  
        }  
    }  
  
    return top;  
}
```

```

    }

    /* If both does not know each other,
    both cannot be the celebrity */
    else {
        top++;
        down--;
    }
}

// Return -1 if no celebrity is found
if(top > down) return -1;

/* Check if the person pointed
by top is celebrity */
for(int i=0; i < n; i++) {
    if(i == top) continue;

    // Check if it is not a celebrity
    if(M[top][i] == 1 || M[i][top] == 0) {
        return -1;
    }
}
}

```

```

        // Return the index of celebrity
        return top;
    }
};

int main() {
    vector<vector<int>> M = {
        {0, 1, 1, 0},
        {0, 0, 0, 0},
        {1, 1, 0, 0},
        {0, 1, 1, 0}
    };

    /* Creating an instance of
    Solution class */
    Solution sol;

    // Function call to find the index of celebrity
    int ans = sol.celebrity(M);

    cout << "The index of celebrity is: " << ans;

    return 0;
}

```

LRU Cache

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
/* Class to implement Nodes  
of a doubly linked list */
```

```
class Node {
```

```
public:
```

```
    int key, val;
```

```
    Node* next;
```

```
    Node* prev;
```

```
// Constructors
```

```
    Node() {
```

```
        key = val = -1;
```

```
        next = prev = NULL;
```

```
    }
```

```
    Node(int k, int value) {
```

```
        key = k;
```

```
        val = value;
```

```
        next = prev = NULL;
```

```
    }
```

```
};
```

// Class implementing LRU cache

class LRUCache {

private:

map <int, Node*> mpp; // Map data structure

int cap; // Capacity

Node* head; // Dummy head pointer

Node* tail; // Dummy tail pointer

/* Private method to delete node

from doubly linked list */

void deleteNode(Node* node) {

// Get the previous and next pointers

Node* prevNode = node-> prev;

Node* nextNode = node-> next;

// Remove pointers to node

prevNode-> next = nextNode;

nextNode-> prev = prevNode;

}

// Private method to insert node after head

```
void insertAfterHead(Node* node) {
```

```
    Node* nextNode = head-> next;
```

```
    head-> next = node;
```

```
    nextNode-> prev = node;
```

```
    node-> prev = head;
```

```
    node-> next = nextNode;
```

```
}
```

```
public:
```

```
// Method to initialise cache with given capacity
```

```
LRUCache(int capacity) {
```

```
    cap = capacity; // Set the capacity
```

```
    mpp.clear(); // Clear the cache
```

```
    head = new Node();
```

```
    tail = new Node();
```

```
// Make the connections
```

```
    head-> next = tail;
```

```
    tail-> prev = head;
```

```
}
```

```
// Method to get the key from cache
```



```

int get(int key_) {
    // Return -1 if it is not present in cache
    if(mpp.find(key_) == mpp.end())
        return -1;

    Node* node = mpp[key_]; // Get the node
    int val = node->val; // Get the value

    // Delete the node
    deleteNode(node);
    /* Insert this node to front
    as it was recently used */
    insertAfterHead(node);

    // Return the stored value
    return val;
}

/* Method to update value if key exists,
otherwise insert the key-value pair */
void put(int key_, int value) {

    // Update the value if key is already present
    if(mpp.find(key_) != mpp.end()) {

```

```

Node* node = mpp[key_]; // Get the node
node->val = value; // Update the value

// Delete the node
deleteNode(node);

/* Insert this node to front
as it was recently used */
insertAfterHead(node);

return;
}

// Check if the capacity limit has reached
if(mpp.size() == cap) {

    // Get the least recently used node
Node* node = tail->prev;

    // Delete node from map
mpp.erase(node-> key);

    // Delete node from doubly linked list

```

```
        deleteNode(node);
    }

    // Create a new node
    Node* newNode = new Node(key_, value);

    // Insert it in map
    mpp[key_] = newNode;

    // Insert in doubly linked list
    insertAfterHead(newNode);
}

};
```

```
int main() {
    // LRU Cache
    LRUCache cache(2);

    // Queries
    cache.put(1, 1);
    cache.put(2, 2);
    cout << cache.get(1) << " ";
    cache.put(3, 3);
```

```

    cout << cache.get(2) << " ";
    cache.put(4, 4);
    cout << cache.get(1) << " ";
    cout << cache.get(3) << " ";
    cout << cache.get(4) << " ";

    return 0;
}

LFU Cache

#include <bits/stdc++.h>

using namespace std;

/* To implement a node in doubly linked
list that will store data items */
struct Node {
    int key, value, cnt;
    Node *next;
    Node *prev;
    Node(int _key, int _value) {
        key = _key;
        value = _value;
        cnt = 1;
    }
};

```

// To implement the doubly linked list

struct List {

int size; // Size

Node *head; // Dummy head

Node *tail; // Dummy tail

// Constructor

List() {

head = new Node(0, 0);

tail = new Node(0,0);

head->next = tail;

tail->prev = head;

size = 0;

}

// Function to add node in front

void addFront(Node *node) {

Node* temp = head->next;

node->next = temp;

node->prev = head;

head->next = node;

temp->prev = node;

size++;

```
}
```

```
// Function to remove node from the list
```

```
void removeNode(Node* delnode) {  
    Node* prevNode = delnode->prev;  
    Node* nextNode = delnode->next;  
    prevNode->next = nextNode;  
    nextNode->prev = prevNode;  
    size--;  
}  
};
```

```
// Class to implement LFU cache
```

```
class LFUCache {  
private:
```

```
// Hashmap to store the key-nodes pairs
```

```
map<int, Node*> keyNode;
```

```
/* Hashmap to maintain the lists
```

```
having different frequencies */
```

```
map<int, List*> freqListMap;
```

```
int maxSizeCache; // Max size of cache
```

```
/* To store the frequency of least  
frequently used data-item */
```

```
int minFreq;
```

```
// To store current size of cache
```

```
int curSize;
```

```
public:
```

```
// Constructor
```

```
LFUCache(int capacity) {
```

```
    // Set the capacity
```

```
    maxSizeCache = capacity;
```

```
    minFreq = 0; // Set minimum frequency
```

```
    curSize = 0; // Set current frequency
```

```
}
```

```
// Method to update frequency of data-items
```

```
void updateFreqListMap(Node *node) {
```

```
    // Remove from Hashmap
```

```
    keyNode.erase(node->key);
```

```
// Update the frequency list hashmap  
freqListMap[node->cnt]->removeNode(node);  
  
// If node was the last node having it's frequency  
if(node->cnt == minFreq &&  
    freqListMap[node->cnt]->size == 0) {  
  
    // Update the minimum frequency  
    minFreq++;  
}  
  
// Creating a dummy list for next higher frequency  
List* nextHigherFreqList = new List();  
  
// If the next higher frequency list already exists  
if(freqListMap.find(node->cnt + 1) !=  
    freqListMap.end()) {  
  
    // Update pointer to already existing list  
    nextHigherFreqList = freqListMap[node->cnt + 1];  
}  
  
// Increment the count of data-item  
node->cnt += 1;
```



```

// Add the node in front of higher frequency list
nextHigherFreqList->addFront(node);

// Update the
freqListMap[node->cnt] = nextHigherFreqList;
keyNode[node->key] = node;
}

// Method to get the value of key from LFU cache
int get(int key) {

    // Return the value if key exists
if(keyNode.find(key) != keyNode.end()) {
    Node* node = keyNode[key]; // Get the node
int val = node->value; // Get the value
updateFreqListMap(node); // Update the frequency

    // Return the value
return val;
}

// Return -1 if key is not found
return -1;

```

```
}
```

```
void put(int key, int value) {  
    /* If the size of Cache is 0,  
    no data-items can be inserted */  
    if (maxSizeCache == 0) {  
        return;  
    }
```

```
    // If key already exists  
    if(keyNode.find(key) != keyNode.end()) {
```

```
        // Get the node  
        Node* node = keyNode[key];
```

```
        // Update the value  
        node->value = value;
```

```
        // Update the frequency  
        updateFreqListMap(node);  
    }
```

```
    // Else if the key does not exist  
    else {
```

```
// If cache limit is reached  
if(curSize == maxSizeCache) {  
  
    // Remove the least frequently used data-item  
    List* list = freqListMap[minFreq];  
    keyNode.erase(list->tail->prev->key);  
  
    // Update the frequency map  
    freqListMap[minFreq]->removeNode(  
        list->tail->prev  
    );  
  
    // Decrement the current size of cache  
    curSize--;  
}  
  
// Increment the current cache size  
curSize++;  
  
// Adding new value to the cache  
minFreq = 1; // Set its frequency to 1  
  
// Create a dummy list
```

```
List* listFreq = new List();

// If the list already exist
if(freqListMap.find(minFreq) !=
    freqListMap.end()) {

    // Update the pointer to already present list
listFreq = freqListMap[minFreq];
}

// Create the node to store data-item
Node* node = new Node(key, value);

// Add the node to dummy list
listFreq->addFront(node);

// Add the node to Hashmap
keyNode[key] = node;

// Update the frequency list map
freqListMap[minFreq] = listFreq;
}
}
};
```

```
int main() {  
    // LFU Cache  
    LFUCache cache(2);  
  
    // Queries  
    cache.put(1, 1);  
    cache.put(2, 2);  
    cout << cache.get(1) << " ";  
    cache.put(3, 3);  
    cout << cache.get(2) << " ";  
    cout << cache.get(3) << " ";  
    cache.put(4, 4);  
    cout << cache.get(1) << " ";  
    cout << cache.get(3) << " ";  
    cout << cache.get(4) << " ";  
  
    return 0;  
}
```